Risk-Aware MCP: Risk Assessment for Model Context Protocol Invocations

Tomer Ovadya
Ben-Gurion University of the Negev
Software and Information Systems Engineering
Beer-Sheva, Israel
ovadyat@post.bgu.ac.il

Asaf Shabtai
Ben-Gurion University of the Negev
Software and Information Systems Engineering
Beer-Sheva, Israel
shabtaia@bgu.ac.il

ABSTRACT

Model Context Protocol (MCP) servers expose tools that allow AI agents to interact directly with organizational assets and services. Even legitimate tool invocations can have substantially different consequences depending on the affected asset, the operation performed, and the scope of its effect.

Such risk does not require malicious intent: misuse may result from overly broad delegation, incorrect tool or parameter selection. This motivates assessing the risk of a concrete MCP invocation before it is executed.

We present a two-stage framework for MCP risk assessment. At design time, a *Static Scanner* constructs a tool–asset risk surface by combining tool impact, organizational asset sensitivity, and blast radius. At runtime, a *Dynamic Scanner* adapts this baseline using invocation-specific arguments, execution context, and previously observed behavior. We evaluate the framework across 16 deployments derived from nine real MCP tool catalogs, covering 262 tools, 221 assets, and 3,615 candidate tool–asset pairs, together with 3,000 live invocations across GitHub, Slack, and Calendar deployments.

CCS CONCEPTS

• **Security and privacy** → **Distributed systems security**; *Access control*; • **Computing methodologies** → *Intelligent agents.*

KEYWORDS

Model Context Protocol; AI agents; risk scoring; security categorization; data classification policy; LLM security; access control

1 INTRODUCTION

AI agents no longer only generate text; they act. They read data, modify files, update databases, and call external services for their users. The Model Context Protocol (MCP) [3] standardizes this exchange: a server publishes tools, and a connected agent discovers and invokes them over JSON-RPC [22]. The public server ecosystem is growing quickly [12], and audits of deployed servers find recurring weaknesses [9]. An agent's decision now lands directly on the systems and assets behind those tools. These tools let agent decisions directly affect organizational assets. MCP standardizes how tools are discovered and invoked, but it does not assess the risk of a specific call. Its annotations describe the tool rather than the asset it affects, so a destructive tool receives the same label whether it targets a temporary file or a production database.

Most MCP and agent security research studies how malicious servers or tools can harm the agents that use them [16, 30]. Other work audits MCP server implementations and exposed capabilities for software weaknesses or excessive privilege [14, 18]. For example, at runtime, MTGuard inspects invocation parameters and observed execution to detect policy violations [10]. These approaches identify vulnerabilities, excessive permissions, or unsafe behavior. They do not estimate the consequence of an authorized invocation from the sensitivity and scope of the asset it affects. Our work provides this estimate.

We study *misuse*: a legitimate capability used beyond what the task requires [2]. It needs no malicious agent and no malicious server. It can start with an operator who states a task loosely, delegates too broadly, or grants excess permission [25] . Studies of MCP applications and servers report broad system access and overprivileged tools [14, 20]. Misuse can also result from model behavior: an agent may select an available but inappropriate tool [36], or recognize a risk in general but fail to account for it during execution [33]. We do not ask why the agent made the call. We assess what the call can do. Reading a build log and reading stored credentials may use the same tool, but the risks are different. We therefore analyze the full invocation: the tool, the asset it affects, the arguments, the execution context, and the agent's recent actions.

Access control decides whether a request is allowed. Role and attribute-based models [13, 28] evaluate permissions under a policy, but they do not normally assign a risk level to an allowed request. This gap also appears in agent frameworks: exposing a tool does not ensure that each call is authorized using its actual argument values [21]. Authentication identifies the caller, and authorization defines what the caller may do. Neither measures the possible consequence of a specific authorized call.

Adding risk to an access-control decision is not new. Early work quantified risk during authorization [6], and later work incorporated risk-adaptive access control into an attribute-based framework [17]. The area has since been widely studied [4]. Recent work extends this idea to agents, using an LLM judge to consider task risk, asset risk, and model uncertainty [7]. However, these models do not define an MCP-specific consequence score for a concrete invocation based on the tool, the sensitivity of the affected asset, and the scope of the call.

To our knowledge, no existing method estimates the consequence of a concrete MCP invocation from the sensitivity and scope of the asset it affects. Many defenses operate at the agent, client, or middleware layer, where they restrict available tools or authorize calls before execution [1, 5, 19, 31, 40]. Server-side authorization has also begun to appear: IGAC checks a proposed call and its payload against the user's declared intent [39]. Other runtime systems return safety scores or decisions such as allow, review, or

block [11, 35, 37]. These outputs indicate whether a call is permitted, suspicious, or requires intervention, but they do not estimate the severity of an authorized call for the specific organizational asset it affects.

Consequently, existing vulnerability-scoring frameworks do not address this problem either. CVSS estimates the severity of vulnerabilities in software, hardware, and firmware [8], while AIVSS extends vulnerability scoring with factors tailored to agentic AI systems [15]. However, both assess vulnerabilities rather than the potential consequence of a concrete MCP operation on an organizational asset. Our approach instead computes this operation-level consequence and uses it as the basis for subsequent risk-aware decisions.

Risk belongs to a tool–asset pair, not to the tool alone. For example,: `read_file` poses little risk when applied to a public document but substantial risk when applied to regulated data. We therefore compute a baseline from three factors: tool impact, asset sensitivity, and *blast radius*, meaning the portion of the asset affected by one call. The call's arguments may change this baseline, while its context and the agent's recent activity may indicate whether the call is unusual.

To address this problem, we introduce **CARMA** (*Continuous Adaptive Risk Model for MCP Actions*), a framework for assessing the risk of MCP operations on organizational assets.

CARMA computes risk in two complementary stages. At design time, its *Static Scanner* establishes a baseline risk for each reachable tool–asset pair by combining tool impact, asset sensitivity, and blast radius. Asset sensitivity is derived from the organization's information-classification policy, following established guidance for mapping information types to impact categories [23, 32]. At request time, the *Dynamic Scanner* adapts this baseline using operation-specific arguments, execution context, and the agent's recent activity. The resulting score can then be compared with organizational policies to support or decline MCP-related actions. In this sense, the score also provides a measurable basis for adjustable autonomy, where control may shift between the agent and a human according to the risk of the current operation [27].

We evaluate CARMA using tool catalogs from real vendor MCP servers together with organizational policies, asset inventories, and concrete invocation scenarios. The evaluation examines both the design-time risk surface produced across tool–asset combinations and the runtime adaptation of these scores as MCP operations are observed.

This paper makes three contributions:

- **Risk-analysis methodology.** We introduce a methodology for assessing the misuse risk of MCP operations by combining tool impact, asset sensitivity, blast radius, runtime arguments, execution context, and the agent's recent activity. Asset sensitivity is derived from organizational policy rather than requiring a fully labeled asset inventory.
- **Risk-assessment framework.** We develop a framework that implements the proposed methodology through complementary Static and Dynamic Scanners and produces interpretable risk scores.
- **MCP-specific evaluation methodology.** We introduce an evaluation methodology for MCP risk assessment that systematically constructs organizational environments around real MCP tool catalogs. For each evaluated MCP, we synthesize representative organizational assets and their sensitivity profiles, derive tool–asset combinations according to the exposed tool capabilities, and craft MCP-specific operation scenarios. We further construct time-ordered dynamic scenarios in which usage patterns, scope, frequency, and behavioral history evolve over time, enabling controlled evaluation of both the static risk factors and the dynamic adaptation of the computed risk.

2 RELATED WORK

MCP Server Security and Risk Assessment. Prior work surveys security risks across the MCP ecosystem [12] and audits open-source MCP servers for software weaknesses, MCP-specific vulnerabilities, and maintainability problems [9]. Several studies examine attacks in which a malicious server, tool description, or external input influences the agent [16, 30, 38]. We study the opposite direction: an agent sends a call that may harm the MCP server or an organizational asset reached through it. Server-to-agent attacks are outside our scope. We assume only that each tool performs the operation represented by its declared interface and asset mapping.

Other work examines the capabilities and weaknesses of MCP servers. Li et al. analyze 2,562 MCP applications and measure their use of file, network, system, and memory APIs [20]. Huang et al. combine static pattern matching with sandboxed execution to detect capabilities such as file access, network requests, and command execution [14]. MCP-in-SoS scans 222 server repositories for code weaknesses, maps them to CWE and CAPEC entries, and derives repository-level risk from the likelihood and severity those catalogs assign [18]. All three assess the server as an artifact, independently of the assets a given installation holds, so the same server receives the same assessment wherever it is deployed.

ConLeash checks whether an MCP call stays within an approved consent boundary [19]. However, it does not estimate the risk or possible impact of a call that is already allowed. Our work addresses this gap by assessing the risk of each invocation from the server side.

Defenses for MCP and tool-using agents. IGAC checks whether an MCP invocation and its arguments remain within the user's stated intent, while MTGuard compares invocation parameters, runtime behavior, and results against safety policies [10, 39]. Progent and MiniScope enforce least-privilege authorization, whereas AgentGuardian learns admissible tool inputs and invocation sequences from normal executions [1, 31, 40]. These systems check whether a tool call follows the user's request, the security policy, or the agent's normal behavior. However, a call can pass all these checks and still cause serious damage. For example, an agent may be allowed to delete files, but deleting a temporary file and deleting a critical production file do not have the same risk; our work measures this difference at the MCP server.

Access control. RBAC assigns permissions through roles, while ABAC evaluates attributes of the caller, asset, requested action, and

environment conditions [13, 28]. These models can express fine-grained policies, but they do not directly estimate the possible consequence of a permitted request.

Risk-adaptive access control adds a calculated risk value to the authorization decision [6]. Kandala et al. place risk within an attribute-based framework, but leave the concrete risk-calculation method outside their scope [17]. Ni et al. use fuzzy rules over subject and object security information to estimate the risk of an access operation [24]. Shaikh et al. dynamically calculate trust and risk for each subject–object pair from the user's past behavior [29]. Atlam et al. survey the wider family of risk-based access-control models [4].

For agentic systems, Fleming et al. use an LLM judge to create a task policy, combine static risk values for its tools and assets, and estimate uncertainty [7]. A limitation is that risk is computed for the whole task and is not updated for each call. For example, reading one database record and exporting a full table may use the same tool risk value. Our work instead scores each MCP invocation using its runtime details.

Risk scoring for MCP and agents. CVSS scores the severity of software vulnerabilities. A study of 196 evaluators applying CVSS v3.1 found differences between evaluators, and a follow-up found that 68% assigned a different severity level to vulnerabilities they had scored before [8, 34]. AIVSS extends vulnerability scoring to agentic AI systems [15]. These frameworks score vulnerabilities rather than concrete tool calls.

FIPS 199 and NIST SP 800-60 classify information types and systems by the potential impact of losing confidentiality, integrity, or availability [23, 32]. These classifications provide an organizational view of consequence that our method uses when assessing the asset affected by a call.

Work closer to our setting studies risk at different stages. MCP-in-SoS scores weaknesses in an MCP server repository [18], while AgenTRIM assigns risk levels to tools and applies stronger checks to riskier ones [5]. STARS scores proposed skill invocations using the request, capability, and runtime context before execution [37]. Owiredu-Ashley instead grades the harm of actions recorded after execution [26]. Together, These methods answer different risk questions: they assess server weaknesses, tool capabilities, invocation safety, or harm after execution. They do not provide a common measure of the consequence to the organization affected by the call. For example, deleting a temporary cache and deleting payroll records may use the same tool and both be irreversible, but their organizational impact is very different. Our work connects each MCP call to the organization's asset policy and estimates its risk before execution.

3 THREAT MODEL

We consider an organizational MCP deployment in which an MCP server exposes tools that allow AI agents to interact with organizational assets and external services. An agent selects a tool and provides concrete arguments, and the MCP server performs the corresponding operation on the reachable asset. We assume that the MCP server and its advertised tools are benign and operate according to their declared functionality. Our protected environment consists of the organizational assets and services reachable through these tools.

Our threat model focuses on *misuse of legitimate MCP capabilities*. Misuse occurs when an MCP operation creates potentially harmful consequences for organizational assets or services, regardless of whether the operation originates from benign error, excessive delegation, compromised behavior, or malicious intent. The operation may still be syntactically valid and supported by the exposed tool. The risk depends on what operation is performed, which asset it affects, how broadly it can affect that asset, and how the capability is exercised at runtime. In contrast to attacks that compromise the MCP server or introduce malicious tools, misuse can arise when a valid MCP operation produces consequences that are inappropriate for the current organizational context.

At design time, this evidence includes the semantics of the exposed tools, the organizational assets they can reach, the sensitivity of those assets, and the scope that an operation can affect. At runtime, additional evidence becomes available from the concrete arguments and execution context.

Common security threats, including malicious MCP servers, tool poisoning, authentication bypass, implementation vulnerabilities, manipulation of organizational policies, and server-to-agent attacks, are outside the scope of this work.

4 METHODOLOGY

4.1 Overview

The CARMA framework separates MCP risk assessment into two layers: *static risk assessment* and *dynamic risk adaptation*. The Static Scanner estimates the potential consequence of capabilities exposed by an MCP server before observing their actual use. The Dynamic Scanner subsequently adapts this baseline using evidence available for a concrete invocation.

This separation reflects the information available at the two stages. Static properties such as operation semantics, organizational sensitivity, and reachable scope can be analyzed when the server and organizational policy are configured. Runtime evidence, such as concrete arguments and previously observed behavior, becomes available only when an invocation occurs. The separation also allows relatively expensive semantic analysis to be performed outside the invocation path while keeping runtime assessment lightweight.

Figure 1 illustrates the overall methodology.

4.2 Static Risk Assessment

The Static Scanner operates over two principal information sources: the MCP tool catalog and organizational asset information. From the MCP server, it collects the exposed tool definitions, including names, descriptions, argument schemas, and available annotations, e.g., through `tools/list` [22]. From the organization, it obtains the asset register and available classification policy or metadata.

Before assigning a score, the scanner determines whether tool t can operate on asset a. Reachability may follow from tool parameters, configured roots, repositories, database scopes, API mappings, or organizational metadata. Pairs that cannot interact are represented as *N/A*, rather than assigned a low risk score.

For every reachable tool–asset pair (t, a), the scanner derives three factors: *tool impact*, *asset sensitivity*, and *blast radius*.

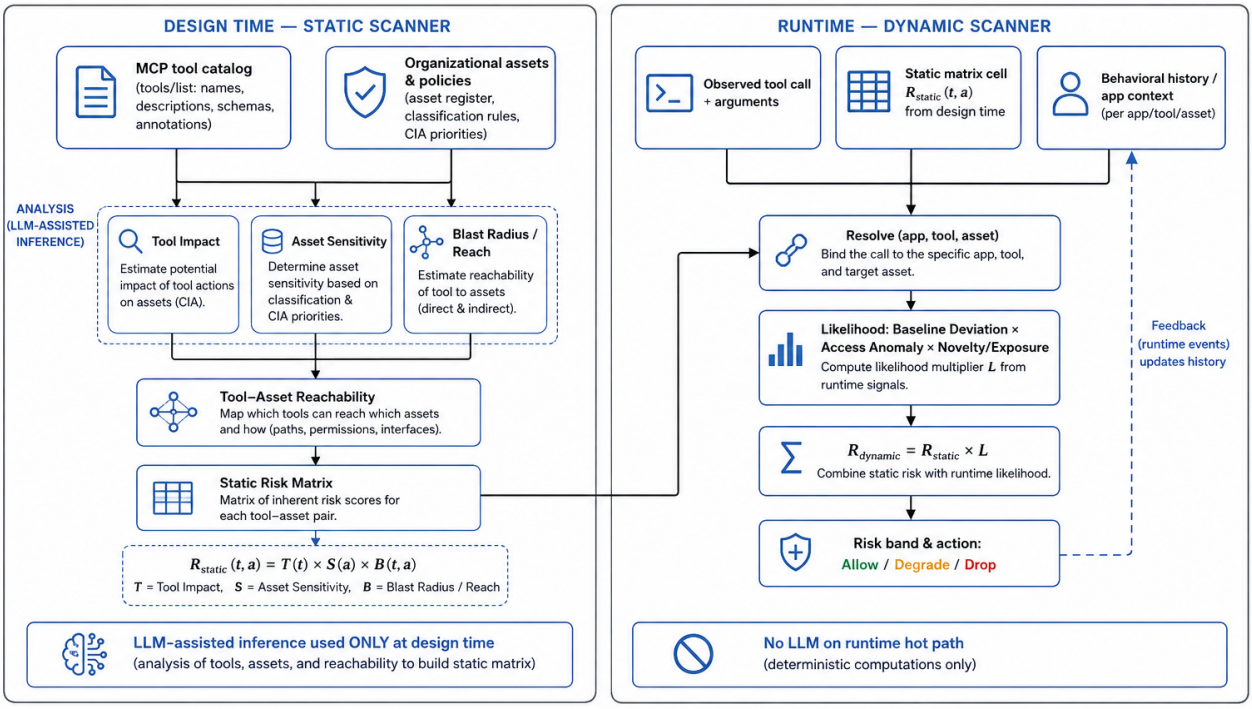


Figure 1: Overview of the proposed MCP risk-assessment methodology. The Static Scanner analyzes MCP tool information and organizational assets to construct a tool–asset risk matrix. At runtime, the Dynamic Scanner adapts the corresponding baseline using invocation-specific and historical evidence.

4.2.1 Tool Impact. Tool impact, denoted by $T(t)$, captures what one invocation of a tool can do, independently of the value of the asset on which it operates. The classification is based on the effective operation rather than only the tool name.

We use an ordinal five-level scale:

- $T = 1$: negligible-effect or liveness operations;
- $T = 2$: metadata or enumeration operations;
- $T = 3$: content-access operations;
- $T = 4$: state-changing operations such as creation or modification;
- $T = 5$: destructive, irreversible, execution, or boundary-crossing operations.

The implementation uses a deterministic operation ladder when the tool semantics map unambiguously to these categories. Tools whose behavior cannot be resolved reliably by the ladder are passed to a semantic classifier. This hybrid design keeps common operations deterministic while retaining coverage for domain-specific tool vocabularies.

4.2.2 Asset Sensitivity. Asset sensitivity, denoted by $S(a)$, captures the potential organizational consequence of exposing, modifying, or destroying asset a. Rather than requiring every asset to carry a manually assigned numerical risk score, the scanner uses the organization's own classification information whenever available.

We use a five-level scale:

- $S = 1$: public or negligible-impact assets;
- $S = 2$: routine internal assets;
- $S = 3$: assets with moderate organizational impact;
- $S = 4$: restricted or high-impact assets;
- $S = 5$: critical assets such as credentials, regulated data, or organizational crown jewels.

When explicit per-asset labels are unavailable, the asset description is interpreted against the organization's data classification policies. The resulting classification is then mapped onto the common five-level scale. This follows the general security-categorization principle of deriving impact from stated organizational confidentiality, integrity, and availability requirements [23, 32].

4.2.3 Blast Radius. Sensitivity captures the importance of an asset, but not how much of it an operation can affect. We therefore define blast radius $B(t, a)$ as the scope reachable through one invocation of tool t on asset a.

We use the following ordinal scale:

- $B = 1$: one object or a very narrow portion of the asset;
- $B = 2$: a small explicitly bounded subset;
- $B = 3$: a broad subset;
- $B = 4$: the complete logical asset or scope;
- $B = 5$: systemic or fan-out impact across multiple assets, recipients, subjects, or dependent systems.

Blast radius is therefore a property of the tool–asset relationship rather than of the tool alone. The same operation may have limited reach when applied to one object and substantially greater reach when applied to an entire collection.

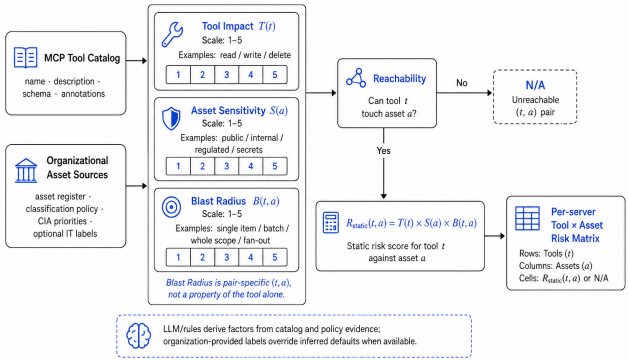


Figure 2: Static Scanner. MCP tool information and organizational asset information are used to estimate tool impact, asset sensitivity, blast radius, and reachability. The output is a per-server tool–asset risk matrix.

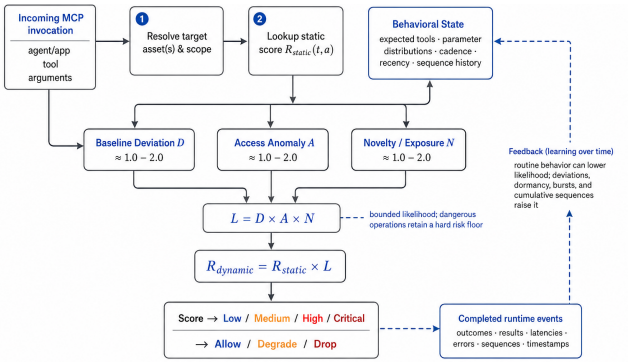


Figure 3: Dynamic Scanner. A concrete invocation is matched with its static tool–asset score and adapted using baseline deviation, access anomaly, novelty/exposure, and previously observed behavior.

4.2.4 Static Risk Score. For each reachable tool–asset pair (t, a), the Static Scanner computes

$$R_{\text{static}}(t, a) = T(t)\, S(a)\, B(t, a). \tag{1}$$

Since all three factors use a 1–5 scale, $R_{\text{static}} \in [1, 125]$. The resulting values form the matrix

$$\mathbf{R}_{\text{static}} = \{R_{\text{static}}(t, a)\}, \tag{2}$$

which represents the design-time risk surface of the MCP server.

The factors retain separate meanings even though they are combined in the final score. A high score can therefore be traced to the operation, the organizational importance of its target, the scope that can be affected, or a combination of the three.

4.3 Dynamic Risk Adaptation

The static score captures the potential consequence of a capability, but two invocations of the same tool on the same asset need not present the same runtime situation. Concrete arguments, requested scope, timing, and previous behavior of the caller provide additional evidence about the current invocation.

For each incoming invocation, the Dynamic Scanner first resolves the target asset and requested scope, and retrieves the corresponding $R_{\text{static}}(t, r)$.

The Dynamic Scanner maintains a behavioral state based on previously completed invocations. This state includes expected tools, parameter distributions, invocation frequency and recency, and sequences of tool use.

The Dynamic Scanner then evaluates request-time evidence using three factors:

Baseline Deviation (D). This factor captures how much the current invocation differs from previously observed behavior, such as changes in tool use, parameter values, frequency, or invocation sequence.

Access Anomaly (A). This factor captures how unusual the requested access is for the target asset, including unusual targets or requested scope.

Novelty and Exposure (N). This factor captures how new or rarely observed the current combination of tool, asset, and parameters is.

Each factor ranges approximately from 1 to 2, where values near 1 represent familiar behavior and larger values represent stronger evidence of unusual or new behavior. We combine these factors into the bounded runtime multiplier:

$$L(c) = D(c)\, A(c)\, N(c), \tag{3}$$

so that $L(c)$ ranges approximately from 1 to 8.

The final assessment is

$$R_{\text{dynamic}}(c) = R_{\text{static}}(t, r)\, L(c). \tag{4}$$

This formulation assigns different responsibilities to the two layers. R_{static} represents the potential consequence of the capability, whereas $L(c)$ represents evidence specific to its current use. Because the runtime factors are bounded below by 1, familiar behavior does not reduce the static risk, while unusual or new behavior can increase the assessment.

The dynamic state evolves as invocations are observed. Information from completed invocations, such as outcomes, errors, latency, sequences, and timestamps, is added to the runtime history. Recurring behavior can become established over time, while changes in scope, frequency, target, parameters, or invocation sequence can raise the assessment again.

4.4 Risk Output

The output of the framework is the graded dynamic assessment $R_{\text{dynamic}}(c)$ together with the factors that contributed to it. An organization may map this value to its own risk bands or use it as input to logging, review, or access-control mechanisms.

For the static characterization reported in Section 7, we use four reporting bands on the 1–125 static scale: *Low* for scores below 17, *Medium* for scores from 17 to 49, *High* for scores from 50 to 99, and *Critical* for scores of 100 or greater. These bands are used to summarize the experimental risk surface and do not prescribe an enforcement policy.

We deliberately separate assessment from enforcement. The framework estimates the risk associated with executing an invocation; the organization determines what action, if any, should follow according to its own risk tolerance.

5 IMPLEMENTATION

CARMA is implemented in Python. Every stage that requires semantic judgment runs at design time against a locally hosted instruction model (Qwen2.5-32B served through Ollama) with greedy decoding and a fixed seed, so a given tool catalog yields a byte-identical risk matrix on re-scan. The runtime path contains no model call at all. This section describes how invocations are captured (Section 5.1), and how the two layers of Section 4 are realized (Sections 5.2 and 5.3).

5.1 Invocation Logging

Both layers are downstream of one capture mechanism. The Static Scanner consumes the tool catalog a server advertises; the Dynamic Scanner consumes the ordered history of invocations against it. We obtain both from the MCP message stream itself rather than from any server-specific API, SDK hook, or agent instrumentation.

The design point is that MCP fixes the *message layer* — every interaction is a JSON-RPC 2.0 request or response — while leaving the *transport* open: a locally spawned server communicates over stdio, and a hosted or vendor-operated server is reached over HTTP, either as HTTP with server-sent events or as streamable HTTP [22]. Capturing at the JSON-RPC layer rather than at a transport therefore yields one implementation that covers both local and web-hosted deployments, and requires no cooperation from the server and no modification to the agent.

We use two collection paths that converge on the same records:

Local (stdio). The server is launched as a subprocess. Requests are written as line-delimited JSON to its standard input, responses are read from its standard output, and its standard error is drained on a separate thread so a chatty server cannot deadlock the pipe. Each request carries a monotonically increasing identifier and responses are matched back to it by that identifier.

Remote (HTTP). The server is fronted by a recording reverse proxy. Every HTTP flow is written as one JSON record carrying request and response bodies, headers, status, and start and end timestamps. Event-stream framing is unwrapped, so the recovered payload is the same JSON-RPC object that the stdio path observes directly.

A parser then reconstructs the interaction from the raw stream alone, pairing requests with responses per session and emitting three tables. The *catalog* table holds one row per tool advertised in a `tools/list` result — name, description, argument and result schemas, and the read-only, destructive, and idempotent annotation hints — and is the input to the static layer. The *calls* table holds one row per invocation — timestamps, elapsed time, tool, argument names, argument values, status or error, and response — and is the input to the dynamic layer. The *sessions* table holds one row per client session, including the negotiated protocol version and the client and server identities exchanged during the handshake.

Two details of the reconstruction matter for risk assessment. First, MCP servers signal failure in two different shapes: a top-level JSON-RPC error object, and a successful response whose result is flagged as an error with the message carried in its content. The parser normalizes both, because a rejected invocation is still evidence about the caller and must not be dropped from the behavioral history. Second, the parser flags structural anomalies that no individual call reveals — sessions that connect but never complete the handshake, repeated identical invocations within a short window, and calls whose latency is far above the observed norm.

The tool catalog is recorded exactly as the server advertises it. Even under the benign-server assumption of Section 3, a tool's declared annotations and prose remain self-reported and imprecise: the protocol defines them as hints, and a catalog written for human readers routinely understates what a tool can reach. The static layer therefore treats the declaration as evidence to be bounded rather than as ground truth, as described next.

5.2 Static Layer

The Static Scanner takes the captured catalog for a server together with the organization's asset register and classification policy, and produces the matrix $\mathbf{R}_{\text{static}}$.

Reachability. The register states which tools may address which assets, and configured roots, repository and database scopes, and schema constraints narrow that further. Pairs left unreachable are emitted as *N/A* and never scored, which is what produces the large non-applicable fraction reported in Section 7.

Tool impact $T(t)$. Impact is resolved by a deterministic operation ladder first, with a semantic classifier used only where the ladder abstains. The ladder applies a strict evidence precedence. MCP tool annotations are consulted first, but only as far as the specification licenses them: the destructive and idempotent hints are meaningful only when the read-only hint is false, and all of them are hints supplied by a possibly untrusted server. An annotation may therefore *bound* a tier — a declared read-only tool cannot exceed content access — but no annotation on its own promotes a tool to the top tier. Within that bound, verb evidence in the tool's name and description decides, and a tool spanning several tiers takes the highest one it reaches. Parameters describe but do not decide: a raw-query parameter, an outbound-send parameter, or an unconstrained path is recorded as a capability flag only, because what a parameter *could* carry is a design-time possibility while what a given call *does* carry is a runtime fact belonging to the dynamic layer. Every classification returns the evidence that produced it, and the scorer that decided each tool is recorded, which is what allows the deterministic and semantic shares to be reported separately.

Asset sensitivity $S(a)$. Sensitivity is derived by classifying each register asset against the organization's own classification policy and mapping the resulting class onto the five-level scale. The policy states adverse impact per class and carries no numbers, so the organization is never asked to supply a per-asset risk score.

Blast radius $B(t, a)$ and banding. Blast radius is estimated per pair rather than per tool. The score is the product of the three factors, and the reporting bands of Section 4.4 are a deterministic function of the primitives — no second model call assigns a band.

An earlier version of the pipeline ran an additional reviewer stage in which an independent model re-derived each primitive

and could override the first. We removed it from the scan path: measured against the deterministic assignment it altered roughly 46% of bands, inflated the Critical band severalfold, and made matrices irreproducible across runs. Its useful contribution was made permanent instead of per-run, by folding its band-level corrections into the deterministic banding rules and its skepticism into the primitive prompts. The reviewer is retained for evaluation only, to measure how often an independent model agrees with the scanner, and is never invoked during a scan.

5.3 Dynamic Layer

The Dynamic Scanner runs in front of the server and must return a verdict before the invocation proceeds. This imposes a hard constraint that shapes the whole implementation: *no model runs on the invocation path.* Every mechanism below is deterministic Python — table lookup, pattern matching, and weighted token overlap — and the design-time output of Section 5.2 reaches it only as artifacts already on disk. A verdict therefore costs no inference, reproduces exactly, and cannot be steered by text an attacker writes into an argument.

Resolving the target asset. A tool is not one action; it is one action per container it can address, and the argument chooses. The resolver turns (t, arguments) into the set of register assets the call reaches, and does so without any per-server configuration. Which argument names a container is discovered from observed traffic by successive elimination: the argument must take values from a bounded set, must be reused across calls rather than minted anew each time, must carry handles rather than natural-language phrases, must be accepted by more than one tool, and its values must map close to one-to-one onto register rows. The verb that enumerates those containers is whichever tool's output contains their identifiers, with the human-readable name read from the text nearest the identifier and a binding accepted only when most occurrences agree. Where nothing enumerates, the resolver falls back to classifying what calls against an identifier returned, counting only tokens that distinguish one container from its siblings.

The resolver is deliberately conservative. An identifier that nothing binds returns the closure over every container the tool can reach, explicitly flagged as unresolved, rather than a low-risk guess — guessing low is how a gate is walked past. It also returns a set rather than a single asset, because an operation such as updating a calendar event genuinely touches the calendar, the event record, and the attendee list at once.

Baseline deviation (D). The behavioral state is a profile per (persona, server) pair built only from that persona's own observed calls: the set of tools it has used, the highest asset sensitivity it has reached, and the mean and standard deviation of its session sizes. There is no labeled training data and no external benchmark. A call deviates when it uses a tool this persona has never used, when it touches an asset more sensitive than any it has touched before — escalating with the size of the gap — or when its session is statistically larger than this persona's own norm. Where no history exists, the implementation reports no signal rather than fabricating one.

Access anomaly (A). This factor asks about the objective rather than about the agent: does the call attack the confidentiality, integrity, or availability objective that the organization named as this asset's primary loss? Both inputs are lookups — the objective a given operation violates comes from the operation classification computed at design time, and the objective an asset protects is the leading axis of its register entry, parsed once per server. We found that treating this as an independent escalator fired on between half and two-thirds of all cells and swamped the bands, so it is applied instead as a *qualifier*: it can withhold an escalation the behavioral signals would otherwise apply, but can never introduce one. A call that is off-axis is left exactly where the static scan placed it.

Novelty and exposure (N). Novelty is carried by the unseen-tool and above-prior-maximum conditions of the persona profile, together with the one signal that inspects call *order*: a read of a highly sensitive asset followed within a short window by an outbound-capable call. Neither call need be individually alarming; the sequence is the evidence, and no per-call or per-cell view can see it.

Composition. The runtime factors of Equation 3 are realized over the discrete reporting bands rather than as continuous multipliers: each signal proposes a band and the final verdict is the most severe proposal, qualified as described above. This preserves the property that matters in Equation 4 — the runtime evidence is bounded below, so familiar behavior can never lower a call beneath its static floor, while unusual behavior raises it. An optional model-based reviewer that inspects concrete arguments can be enabled for offline analysis; when it is not run this is recorded explicitly rather than being treated as a clean verdict, and it is disabled by default so that the invocation path stays model-free.

Every verdict retains each contributing signal separately alongside the final band — the static floor, the deviation, the sequence, the objective qualification — so that an assessment can be explained in terms of the evidence that produced it rather than presented as a single opaque number. The per-stage timings recorded by both layers provide the measurement points for the operational properties evaluated in Section 6.4.

6 EVALUATION

Our evaluation studies whether the proposed method produces different and explainable risk scores across MCP environments, and whether the dynamic layer can be used on real invocation traces.

No existing benchmark evaluates MCP risk at the granularity considered in this work. Prior MCP and agent-security approaches assess different objects, including server-level risk, tool-level risk, and request-level safety [5, 18, 37], and therefore do not provide a comparable reference for tool–asset or invocation-level risk estimation.

More fundamentally, there is no established ground-truth risk score for an authorized MCP invocation against which such estimates can be calibrated. Consequently, our evaluation does not treat risk estimation as a regression problem against a predefined "correct" score. Instead, we evaluate whether the proposed methodology produces consistent risk ordering, responds systematically to changes in individual risk factors, and adapts appropriately to runtime behavioral context.

6.1 Evaluation Settings

We evaluate the framework on MCP server deployments from several application domains. For each deployment, we obtain the tool catalog exposed through the MCP interface and use the organization's data-classification policy to determine the sensitivity of the assets that the server can access. Some deployments use the same MCP tool catalog under different organizational settings, allowing us to examine how organizational context affects the resulting risk assessment.

For the static evaluation, the framework builds a tool–asset matrix for each deployment and assigns a risk score to each reachable pair.

For the dynamic evaluation, we use invocation logs collected from the deployments. These logs contain time-ordered MCP calls, including their concrete arguments, targets, operation sequences, and previous invocation history. They provide the runtime information used by the dynamic layer.

6.2 Static Assessment

The static evaluation studies the risk surface produced before runtime behavior is observed. We examine the distribution of reachable and non-applicable tool–asset pairs, the resulting risk bands, and representative high-risk combinations across different MCP domains.

The three-factor decomposition further permits component-level inspection. In particular, we examine how tool impact, asset sensitivity, and blast radius combine across different MCP environments, and whether high-risk assessments can be traced to the evidence underlying these factors.

We evaluated 16 server deployments drawn from 9 distinct MCP tool catalogs. Each catalog was captured from a running server through the protocol's tool-listing interface [22] and used as exposed by the server. GitHub, Slack, and calendar catalogs were each instantiated under two organizations, while the filesystem catalog was instantiated under five organizations with different asset registers.

In total, the scan covers 262 tools, 221 registered assets, and 3,615 candidate tool–asset cells. Asset sensitivity was derived from the organizational classification policy (synthesized using NIST format [32]).

6.3 Dynamic Assessment

The runtime evaluation studies the feasibility of applying the dynamic layer to real MCP invocation traces. We exercise concrete deployments with time-ordered calls and record the information required by f_{dyn}, including arguments, targets, call frequency, and invocation history.

The preliminary evaluation focuses on whether the runtime evidence can be collected and scored efficiently on the invocation path. A full assessment against independently annotated misuse traces is left for subsequent evaluation.

6.4 Additional Properties

We additionally evaluate operational properties that do not require a reference risk score.

Computational Cost. Because the Static Scanner uses LLM-based analysis to derive the static risk factors, we measure the total computational cost required to analyze a given MCP server and construct its tool–asset risk matrix.

Runtime Latency. For the Dynamic Scanner, we measure the per-invocation latency introduced by risk computation, i.e., the time required to compute the dynamic risk score for an incoming MCP request before execution.

7 PRELIMINARY RESULTS

Currently, we report only the results of the Static Scanner. This section presents the static risk surface produced by our framework from real MCP tool catalogs.

7.1 Static Assessment Results

Table 1 shows that 1,182 of the 3,615 candidate cells (33%) were scored, while 2,433 (67%) were classified as not applicable. Thus, the framework classifies the majority of the Cartesian tool–asset space as non-applicable, substantially reducing the set of tool–asset pairs for which a risk score is produced.

The proportion varies substantially by server structure. For example, the five-tool SQL deployment leaves only 18 of 55 cells unreachable, whereas the 30-tool OpenBB deployment leaves 114 of 150 unreachable because many of its tools expose narrow, specialized functionality.

Among the reachable cells, 444 are Low, 458 Medium, 242 High, and 38 Critical. High and Critical cells together account for approximately 24% of the scored surface and 7.7% of the complete matrix.

Most tool-impact assignments are deterministic. Across the 151 distinct tools, the deterministic operation ladder assigned impact to 127 tools (84%), while semantic analysis was required for the remaining 24. The split reflects differences in catalog vocabulary. Tools with common operation semantics such as `get`, `search`, and `list` are usually resolved directly, whereas domain-specific verbs such as `fork`, `merge`, or `push` more often require semantic interpretation.

This result is operationally useful because semantic inference is concentrated on the ambiguous portion of the tool surface rather than being required for every tool.

7.1.1 Highest-Risk Combinations. Table 2 reports the highest-scoring tool–asset pair in each deployment. The resulting combinations illustrate the interaction between the three static factors: high scores arise when consequential operations, sensitive organizational assets, and broad scope coincide.

The organization-specific component is visible in the filesystem deployments: the highest-scoring `write_file` pair changes with the asset that each organization's policy considers most consequential, including payment data, security keys, patient information, legal matter files, and unreleased media.

The public-data deployments provide the opposite case. The `finance_tools`, `sec_edgar`, and `yahoo_finance` deployments produce no High or Critical cells across their 273 candidate cells. This difference arises despite using the same scoring methodology, illustrating the role of organizational asset context rather than tool semantics alone.

The scan also grades every input parameter for how much stored data it alone can reach, on a five-level scale, independently of whether

Table 1: Static scan by deployment. *Cells* denotes the Cartesian tool–asset space; *Scored* is the subset judged reachable.

							Scored cells by band			
Kind	Deployment	Tools	Assets	Cells	Scored	N/A	Low	Med.	High	Crit.
Filesystem	fs_fintech_fs	14	18	252	103	149	39	36	24	4
Filesystem	fs_medical_clinic_fs	14	16	224	86	138	27	37	22	0
Filesystem	fs_corp_filesystem	14	16	224	87	137	36	26	21	4
Filesystem	fs_law_firm_fs	14	15	210	89	121	33	43	13	0
Filesystem	fs_media_studio_fs	14	16	224	111	113	63	36	12	0
Github	github_real	26	20	520	139	381	39	48	52	0
Github	github_helios	26	19	494	126	368	29	46	47	4
Chat	slack_real	16	20	320	101	219	37	48	6	10
Chat	slack_vireo	16	15	240	62	178	18	38	5	1
Calendar	calendar_real	13	16	208	68	140	22	27	14	5
Calendar	calendar_aurora	13	17	221	71	150	14	33	18	6
SQL	sqlite_cbg_sqlite	5	11	55	37	18	13	13	7	4
Finance	finance_tools	17	6	102	27	75	18	9	0	0
Finance	openbb	30	5	150	36	114	23	12	1	0
Finance	sec_edgar	21	6	126	27	99	24	3	0	0
Finance	yahoo_finance	9	5	45	12	33	9	3	0	0
Total	**16 deployments**	**262**	**221**	**3,615**	**1,182**	**2,433**	**444**	**458**	**242**	**38**

Table 2: Highest-scoring tool–asset pair in each deployment.

Deployment	Highest-scoring tool–asset pair	$S \cdot B \cdot T$	Score
fs_fintech_fs	write_file × card-vault	$5 \cdot 5 \cdot 4$	100
fs_corp_filesystem	write_file × security-keys	$5 \cdot 5 \cdot 4$	100
fs_medical_clinic_fs	write_file × patient-charts	$5 \cdot 4 \cdot 4$	80
fs_law_firm_fs	write_file × matter-files	$5 \cdot 4 \cdot 4$	80
fs_media_studio_fs	write_file × unreleased-imagery	$4 \cdot 4 \cdot 4$	64
github_helios	push_files × helios-scada-gateway	$5 \cdot 5 \cdot 4$	100
github_real	fork_repository × infra-config	$4 \cdot 5 \cdot 4$	80
slack_real	conversations_leave × incident-response	$5 \cdot 5 \cdot 5$	125
slack_vireo	usergroups_users_update × usergroup-membership	$4 \cdot 5 \cdot 5$	100
calendar_real	manage-accounts × connected-account-config	$5 \cdot 5 \cdot 5$	125
calendar_aurora	delete-event × aurora-exec	$5 \cdot 5 \cdot 5$	125
sqlite_cbg_sqlite	write_query × api_keys	$5 \cdot 5 \cdot 5$	125
finance_tools	calculate × expression-evaluator	$4 \cdot 3 \cdot 3$	36
openbb	install_skill × server-capability-install	$4 \cdot 4 \cdot 4$	64
sec_edgar	get_recommended_tools × concept-catalogs	$3 \cdot 3 \cdot 3$	27
yahoo_finance	get_historical_stock_prices × research-query-pattern	$3 \cdot 3 \cdot 3$	27

the parameter carries a numeric quantity: an identifier parameter has no magnitude, but it still selects what is opened, so it still receives a grade. On the GitHub, Slack, and Calendar catalogs used in the dynamic evaluation below, this classification reached full coverage. All 303 parameters (130, 63, and 110, respectively) were graded, with none left ungraded.

7.2 Dynamic Assessment Results

We scored the dynamic layer on the GitHub, Slack, and Calendar deployments using the 3,000 live invocations introduced in §6.1. For each server, we used the first 500 calls to establish a per-tool usage baseline and scored the following 500. Each score combines two runtime signals: one that can only narrow the static estimate,

based on how far a call's arguments sit from that deployment's typical range for the same parameter, and one that moves the estimate up or down based on how often this deployment calls the invoked tool relative to the rest of its catalog. Across the 1,500 scored calls, the combined runtime adjustment had a median of 0.89 on GitHub (range 0.72–0.91), 0.89 on Slack (0.69–0.95), and 0.92 on Calendar (0.73–0.94). No scored call increased the static estimate. Scoring itself is inexpensive: 47–53 microseconds per call across the three servers, or 19,000–21,000 calls per second, with no model invocation and no network access on the request path. This answers the runtime-latency question posed in §6.4.

This evaluation shows that the runtime layer can be computed cheaply and behaves as specified on real traffic. It does not show that the layer separates misuse from ordinary use: every one of the 3,000 invocations is a benign call from normal deployment activity, so nothing here establishes detection performance against misuse. §6.3 already identifies that evaluation as subsequent work. The upward adjustment is still exercised: three tools absent from this evaluation's calls (`delete-event` on Calendar, `create_or_update_file` on GitHub, `conversations_add_message` on Slack) each receive a 1.28× escalation when checked directly against the same warmed baseline. The escalation path activates on genuinely unusual tool choices, not only on the discount side.

8 CONCLUSION

Authorization answers whether an MCP invocation is permitted; it does not express how consequential that permitted invocation may be. We presented a server-side framework for estimating this risk before execution.

The framework separates assessment into two layers. At design time, the Static Scanner constructs a risk surface over reachable tool–asset pairs using three interpretable factors: tool impact, organizational asset sensitivity, and blast radius. Asset sensitivity is grounded in the organization's own classification policy, following established security-categorization principles [23, 32]. At runtime, the Dynamic Scanner adapts the corresponding baseline using invocation-specific context and previously observed behavior.

Our preliminary study covers 16 deployments of 9 real MCP tool catalogs, comprising 262 tools, 221 organizational assets, and 3,615 candidate tool–asset pairs. The framework identifies 67% of this Cartesian space as not applicable, while High and Critical pairs account for 7.7% of the full matrix. A deterministic operation ladder resolves 84% of distinct tool-impact assignments, limiting semantic inference to the ambiguous portion of the tool surface. We additionally collected 3,000 live invocations across three provisioned MCP deployments to exercise the runtime path.

A central property of the framework is that each assessment remains decomposable. Rather than presenting an organization with only an opaque number, the score can be traced to the operation, the organization's own asset classification, the reachable scope, and the runtime evidence that contributed to it.

The current results characterize the risk surface produced by the framework rather than agreement with an independent universal notion of MCP risk. Establishing agreement with organization-assigned severities and evaluating the dynamic layer against independently annotated invocation traces are therefore important directions for further evaluation.

REFERENCES

[1] Nadya Abaev, Denis Klimov, Gerard Levinov, David Mimran, Yuval Elovici, and Asaf Shabtai. 2026. AgentGuardian: Learning Access Control Policies to Govern AI Agent Behavior. *arXiv preprint arXiv:2601.10440* (2026).

[2] James P. Anderson. 1980. *Computer Security Threat Monitoring and Surveillance.* Technical Report. James P. Anderson Co., Fort Washington, PA.

[3] Anthropic. 2024. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol. Accessed 2026-07-30.

[4] Hany F. Atlam, Muhammad Ajmal Azad, Madini O. Alassafi, Abdulrahman A. Alshdadi, and Ahmed Alenezi. 2020. Risk-Based Access Control Model: A Systematic Literature Review. *International Journal of Computer Network and Information Security* 12, 5 (2020).

[5] Roy Betser, Shamik Bose, Amit Giloni, Chiara Picardi, Sindhu Padakandla, and Roman Vainshtein. 2026. AgenTRIM: Tool Risk Mitigation for Agentic AI. *arXiv preprint arXiv:2601.12449* (2026).

[6] Pau-Chen Cheng, Pankaj Rohatgi, Claudia Keser, Paul A. Karger, Grant M. Wagner, and Angela Schuett Reninger. 2007. Fuzzy Multi-Level Security: An Experiment on Quantified Risk-Adaptive Access Control. In *IEEE Symposium on Security and Privacy (S&P).* https://doi.org/10.1109/SP.2007.21

[7] Charles Fleming, Ashish Kundu, and Ramana Kompella. 2025. Uncertainty-Aware, Risk-Adaptive Access Control for Agentic Systems using an LLM-Judged TBAC Model. *arXiv preprint arXiv:2510.11414* (2025).

[8] Forum of Incident Response and Security Teams (FIRST). 2023. *Common Vulnerability Scoring System Version 4.0: Specification Document.* Technical Report. Forum of Incident Response and Security Teams. https://www.first.org/cvss/v4.0/specification-document Document Version 1.2.

[9] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Bram Adams, and Ahmed E. Hassan. 2025. Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers. *arXiv preprint arXiv:2506.13538* (2025).

[10] Ping He, Yuexiang Xie, Yaliang Li, and Shouling Ji. 2026. Hybrid Analysis for Secure MCP Tool Use in LLM Agents. *arXiv preprint arXiv:2607.25297* (2026).

[11] Elias Hossain, Md Mehedi Hasan Nipu, Tasfia Nuzhat Ornee, Rajib Rana, and Niloofar Yousefi. 2026. NEXUS: Structured Runtime Safety for Tool-Using LLM Agents. *arXiv preprint arXiv:2607.19356* (2026).

[12] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2026. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *ACM Transactions on Software Engineering and Methodology* (2026). https://doi.org/10.1145/3796519 arXiv:2503.23278.

[13] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations.* Technical Report NIST Special Publication 800-162. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-162

[14] Charoes Huang, Xin Huang, and Amin Milani Fard. 2026. Auditing MCP Servers for Over-Privileged Tool Capabilities. *arXiv preprint arXiv:2603.21641* (2026).

[15] Ken Huang, Michael Bargury, Vineeth Sai Narajala, Bhavya Gupta, and Tim Marple. 2026. *AIVSS Scoring System for OWASP Agentic AI Core Security Risks.* Technical Report. OWASP Foundation. https://aivss.owasp.org/assets/publications/AIVSS%20Scoring%20System%20For%20OWASP%20Agentic%20AI%20Core%20Security%20Risks%20v0.8.pdf Version 0.8.

[16] Invariant Labs. 2025. MCP Security Notification: Tool Poisoning Attacks. https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks. Accessed 2026-07-30.

[17] Savith Kandala, Ravi Sandhu, and Venkata Bhamidipati. 2011. An Attribute Based Framework for Risk-Adaptive Access Control Models. In *Sixth International Conference on Availability, Reliability and Security (ARES).* https://doi.org/10.1109/ARES.2011.41

[18] Pratyay Kumar, Miguel Antonio Guirao Aguilera, Srikathyayani Srikanteswara, Satyajayant Misra, and Abu Saleh Md Tayeen. 2026. MCP-in-SoS: Risk Assessment Framework for Open-Source MCP Servers. *arXiv preprint arXiv:2603.10194* (2026).

[19] Ying Li, Yanju Chen, Peiran Wang, Issac Khabra, Faysal Hossain Shezan, Yu Feng, and Yuan Tian. 2026. Options, Not Clicks: Lattice Refinement for Consent-Driven MCP Authorization. *arXiv preprint arXiv:2605.11360* (2026).

[20] Zhihao Li, Kun Li, Boyang Ma, Minghui Xu, Yue Zhang, and Xiuzhen Cheng. 2025. We Urgently Need Privilege Management in MCP: A Measurement of API Usage in MCP Ecosystems. In *IEEE International Conference on Mobile Ad-Hoc and Smart Systems (MASS).* arXiv:2507.06250.

[21] David Mellafe Zuvic. 2026. Capability Gates Are Not Authorization: Confused-Deputy Failures in LLM Agent Frameworks. *arXiv preprint arXiv:2606.28679* (2026).

[22] Model Context Protocol. 2026. Model Context Protocol Specification. https://modelcontextprotocol.io/specification. Accessed 2026-07-30.

[23] National Institute of Standards and Technology. 2004. *Standards for Security Categorization of Federal Information and Information Systems*. Technical Report FIPS PUB 199. U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.199

[24] Qun Ni, Elisa Bertino, and Jorge Lobo. 2010. Risk-Based Access Control Systems Built on Fuzzy Inferences. In *Proceedings of the 5th ACM Symposium on Information, Computer and Communications Security (ASIACCS '10)*. Association for Computing Machinery, 250–260. https://doi.org/10.1145/1755688.1755719

[25] OWASP Foundation. 2025. LLM06:2025 Excessive Agency. OWASP Top 10 for Large Language Model Applications. https://genai.owasp.org/llmrisk/llm06-excessive-agency/ Accessed 2026-08-02.

[26] Harry Owiredu-Ashley. 2026. Beyond Attack-Success Rate: Action-Graded Severity Scale for Tool-Using AI Agents. *arXiv preprint arXiv:2607.07474* (2026).

[27] David V. Pynadath, Paul Scerri, and Milind Tambe. 2002. Towards Adjustable Autonomy for the Real World. *Journal of Artificial Intelligence Research* 17 (2002), 171–228. https://doi.org/10.1613/jair.1037

[28] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. 1996. Role-Based Access Control Models. *Computer* 29, 2 (1996), 38–47. https://doi.org/10.1109/2.485845

[29] Riaz Ahmed Shaikh, Kamel Adi, and Luigi Logrippo. 2012. Dynamic Risk-Based Decision Methods for Access Control Systems. *Computers & Security* 31, 4 (2012), 447–464. https://doi.org/10.1016/j.cose.2012.02.006

[30] Jiawen Shi, Zenghui Yuan, Guiyao Tie, et al. 2025. Prompt Injection Attack to Tool Selection in LLM Agents. *arXiv preprint arXiv:2504.19793* (2025).

[31] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. *arXiv preprint arXiv:2504.11703* (2025).

[32] Kevin Stine, Rich Kissel, William C. Barker, Jim Fahlsing, and Jessica Gulick. 2008. *Guide for Mapping Types of Information and Information Systems to Security Categories*. Technical Report NIST Special Publication 800-60 Vol. 1, Rev. 1. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-60v1r1

[33] Yuzhi Tang, Tianxiao Li, Elizabeth Li, Chris J. Maddison, Honghua Dong, and Yangjun Ruan. 2025. LM Agents May Fail to Act on Their Own Risk Knowledge. *arXiv preprint arXiv:2508.13465* (2025).

[34] Julia Wunder, Andreas Kurtz, Christian Eichenmüller, Freya Gassmann, and Zinaida Benenson. 2024. Shedding Light on CVSS Scoring Inconsistencies: A User-Centric Study on Evaluating Widespread Security Vulnerabilities. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1102–1121. https://doi.org/10.1109/SP54263.2024.00058

[35] Chenglin Yang. 2026. AgentTrust: Runtime Safety Evaluation and Interception for AI Agent Tool Use. *arXiv preprint arXiv:2605.04785* (2026).

[36] Chenlong Yin, Zeyang Sha, Shiwen Cui, Changhua Meng, and Zechao Li. 2025. The Reasoning Trap: How Enhancing LLM Reasoning Amplifies Tool Hallucination. *arXiv preprint arXiv:2510.22977* (2025).

[37] Guijia Zhang, Shu Yang, Xilin Gong, and Di Wang. 2026. STARS: Skill-Triggered Audit for Request-Conditioned Invocation Safety in Agent Systems. *arXiv preprint arXiv:2604.10286* (2026).

[38] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. 2026. Parasites in the Toolchain: A Large-Scale Analysis of Attacks on the MCP Ecosystem. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE. https://doi.org/10.1109/SP63933.2026.00154

[39] Genliang Zhu and Chu Wang. 2026. Intent-Governed Tool Authorization for AI Agents. *arXiv preprint arXiv:2606.22916* (2026).

[40] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, et al. 2025. MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents. *arXiv preprint arXiv:2512.11147* (2025).